الجامعة المصرية اليابانية للعلوم والتكنولوجيا
E-JUST
Egypt - Japan University of Science and Technology
エジプト日本科学技術大学

CryptoCare

Secure Healthcare Portal

TEAM MEMBERS

Jana Yousri Attia
Student ID: 320230014

Amnah Hossam Elkheshn
Student ID: 320230082

Yassin Alaa Helmy
Student ID: 32023098

Course: Cryptography and Cryptanalysis— CNC 321

Instructor: Dr. Muhammad Hataba

Egypt-Japan University for Science and Technology

Table of Contents

1. Introduction & Problem Statement

Cryptography secures information by transforming it into an unreadable form unless the correct key is known. The goals are confidentiality, integrity, and authenticity. You get there through symmetric ciphers using shared secret keys, asymmetric systems built on public/private pairs, and supporting primitives like signatures and hashes.

Protection is needed in two places. **Data in transit** moving across vulnerable networks and **data at rest** where stored records can be exposed through theft or insider misuse. Healthcare systems face real and growing threats here like Ransomware locking hospital networks, credential attacks, man-in-the-middle exploits on unsecured APIs. Regulations like HIPAA and GDPR make this worse because correct cryptographic deployment isn't optional… it's a legal requirement.

Our project, **CryptoCare**, builds a secure healthcare portal from scratch. Integrating:

- Two symmetric block ciphers (XTEA and IDEA in CBC mode) for patient data storage and transmission.
- An ElGamal asymmetric system for session-key exchange and digital signatures.
- A PKI Certificate Authority issuing and verifying X.509 certificates for authentication.
- A forward-looking Ring-LWE post-quantum scheme for quantum-resistant tokens.

2. Project Objectives

Our main objective is to create a complete cryptographic system **CryptoCare**, without relying on external cryptographic libraries. The five core objectives below that guided our implementation:

- **Symmetric encryption for data at rest:** We implemented two block ciphers XTEA and IDEA both in CBC mode with PKCS#7 padding to protect patient records stored on the server. Choosing two structurally different algorithms (XTEA uses XOR and bit-shifts; IDEA uses three algebraically incompatible group operations) allowed us to compare their design philosophies and performance side by side.
- **Asymmetric encryption for key exchange:** We implemented ElGamal encryption and digital signatures to solve the key-distribution problem. ElGamal is used exclusively for session-key exchange and physician authorisation signatures, while bulk data encryption is handed off to the faster symmetric ciphers.
- **Multiparty system with certificate based authentication:** We built a Certificate Authority that issues and revokes X.509-style certificates for all parties (server, patients and doctors). The authentication of each party is done using these certificates and a TLS handshake before establishing an encrypted session with support for mid session key rotation.
- **Performance analysis:** We measured all the implemented algorithms for encryption and decryption speed, key generation latency, memory usage, as well as ciphertext size overhead enabling us to analyze the hybrid design choices made throughout the system.

- **Post-quantum authentication**: We implemented a Ring-LWE scheme inspired by CRYSTALS-Kyber, to generate quantum resistant authentication tokens. Despite the toy parameters (n=16, q=257) not being production-secure, the implementation shows us the mechanism and why post-quantum cryptography matters for sensitive data.

3. Literature Review

Building CryptoCare required grounding each design decision in established research. The ten sources below from IEEE, ACM, Springer, and NIST cover the six areas most relevant to our implementation:

Symmetric Encryption

[1] **Needham, R. & Wheeler, D. (1997).** Tea Extensions. Technical Report, University of Cambridge Computer Laboratory. The original XTEA report corrected TEA's related-key and equivalent-key weaknesses by replacing the linear key schedule with a cyclic shift-based one. This was our primary reference for implementing the 64-round Feistel structure and DELTA-based sum schedule.

[2] **Lai, X. & Massey, J. L. (1991).** "A Proposal for a New Block Encryption Standard." Advances in Cryptology EUROCRYPT 1990, Lecture Notes in Computer Science, vol. 473, pp. 389–404, Springer. The foundational paper for IDEA, introducing three algebraically incompatible group operations (XOR, addition mod 2^{16}, multiplication mod 65537) as the basis for confusion and diffusion. Our IDEA implementation follows this specification exactly, including the 52-subkey schedule derived by rotating the 128-bit key left by 25 bits.

[3] **Dudhat, H. et al. (2025).** "A Comparative Analysis of Symmetric and Asymmetric Cryptographic Algorithms: Performance, Security, and Versatility." ICT Systems and Sustainability, Lecture Notes in Networks and Systems, vol. 1160, Springer. Provides a structured framework for comparing symmetric and asymmetric algorithms on key size, throughput, and computational cost. Confirmed our design choice to use symmetric ciphers for bulk record encryption, reserving asymmetric operations for session-key exchange only.

Asymmetric Encryption & Digital Signatures

[4] **ElGamal, T. (1985).** "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms." IEEE Transactions on Information Theory, vol. IT-31, no. 4, pp. 469–472. The original ElGamal paper, establishing both the encryption scheme and the signature scheme on the hardness of the discrete logarithm problem. Our implementation follows the encryption and signing equations directly, using safe primes ($p = 2q+1$) to eliminate small-subgroup attacks that the original paper acknowledged as a concern.

[5] **Jintcharadze, E. & Iavich, M. (2020).** "Hybrid Implementation of Twofish, AES, ElGamal and RSA Cryptosystems." 2020 IEEE East-West Design & Test Symposium (EWDTS), pp. 1–5. Benchmarks hybrid cryptographic architectures combining symmetric and asymmetric algorithms, showing that wrapping a symmetric session key in an asymmetric envelope consistently outperforms full asymmetric encryption for payload data. This validated the hybrid design at the core of CryptoCare.

Key Management & Certificate Authorities

[6] **NIST SP 800-57 Part 1 Rev. 5. (2020).** Recommendation for Key Management: Part 1 — General. National Institute of Standards and Technology. doi: 10.6028/NIST.SP.800-57pt1r5. The authoritative federal guidance on cryptographic key lifecycle management, covering generation, distribution, storage, rotation, and revocation. Our session-key rotation logic and certificate revocation list are structured around the key-state transitions defined in this document.

[7] **NIST SP 800-57 Part 3 Rev. 1. (2015).** Recommendation for Key Management: Part 3 — Application-Specific Key Management Guidance. National Institute of Standards and Technology. — Extends Part 1 to application-layer PKI use cases, including how CAs should issue, verify, and revoke X.509 certificates. Directly informed the design of our CertificateAuthority class and the certificate body format (serial number, validity window, issuer signature).

Cryptographic Performance Studies

[8] **Performance Analysis of Cryptographic Algorithms for Selecting Better Utilization on Resource Constraint Devices. (2019).** IEEE Xplore, doi: 10.1109/ICCIDS.2019.8631957. Benchmarks AES, RC4, Blowfish, 3DES, DSA, and ElGamal across key size, data block size, and encryption time, confirming that asymmetric algorithms are orders of magnitude slower than symmetric ones for bulk data. Our benchmarking module uses the same metrics — throughput (MB/s), key-generation latency, and memory footprint — to report XTEA and IDEA performance comparably.

Multi-Party Secure Communication

[9] **NIST FIPS 197. (2023).** Advanced Encryption Standard (AES), updated May 2023. National Institute of Standards and Technology. doi: 10.6028/NIST.FIPS.197-upd1. While AES is not used in our implementation, this standard provides the conceptual baseline for block cipher design (128-bit block, CBC/GCM modes, key-schedule design) against which we contextualize XTEA and IDEA. The NIST framework for multi-party secure sessions built on block ciphers informed our TLS-style handshake structure.

Post-Quantum Cryptography

[10] **Lyubashevsky, V., Peikert, C. & Regev, O. (2013).** "On Ideal Lattices and Learning with Errors over Rings." Journal of the ACM, vol. 60, no. 6, pp. 43:1–43:35. ACM. doi: 10.1145/2535925. Introduced Ring-LWE and proved it is as hard as worst-case problems on ideal lattices, including under quantum reduction. This is the theoretical foundation for our Ring-LWE bonus implementation. The polynomial ring R_q = Z_q[x]/(x^n + 1), the small-error distribution, and the key-generation and encryption equations in our lwe.py follow the constructions in this paper.

Research Gap Addressed by This Project

No study we found combines XTEA and IDEA as a comparative symmetric pair, wraps them in an ElGamal hybrid with a full custom PKI, and extends the stack with a from-scratch Ring-LWE layer all within a single, coherent application domain. CryptoCare fills this gap by offering a fully self-contained, end-to-end educational implementation where every algorithm, every subkey, and every certificate field is computed by hand-written code, making the mathematical structure of each primitive directly observable and testable.

4. Methodology

Implementation Approach

Every cryptographic primitive in CryptoCare was implemented from scratch in Python using only the standard library without using external cryptography packages.

We simply divided the codebase into self contained modules: *src/symmetric/* contains XTEA and IDEA; *src/asymmetric/* contains ElGamal; *src/ca/* contains the Certificate Authority; *src/pqc/* contains Ring-LWE; and *src/app/* wires everything into the healthcare portal with a TLS style session layer.

The only external building blocks relied upon from outside the implementation are Python's built-in *pow()* for modular exponentiation and *os.urandom()* for cryptographically random bytes, everything else including Miller-Rabin primality, extended GCD, modular inverse, safe-prime generation were written by hand in *src/utils/math_utils.py*.

Handwritten Verification

Before writing any code, each algorithm was traced by hand using small numeric examples to confirm our understanding of the mathematical structure. The attached handwritten verification document covers all four algorithms:

XTEA: We first manually computed one full round with K = [1, 2, 3, 4] and plaintext block (v_0 = 1, v_1 = 2). Our step by step arithmetic using the shift, XOR, and addition operations yielded v_0 = 36 after the first half round and v_1 = 2,654,436,350 after the second with the sum updated to 0x9E3779B9. These values were then matched to the *encrypt_block_raw(1, 2)* output of the code.

IDEA: We verified the 3 group operations individually: multiplicative inverse of 3 mod 65537 (= 21846, confirmed by 3 × 21846 = 65538 ≡ 1), additive inverse of 5 mod 65536 (= 65531)

Then we traced one full round with a 16-zero-byte key and plaintext (X_1=1, X_2=2, X_3=3, X_4=4). All intermediate values (a, b, c, d, e, f, g, h) were computed and checked against *encrypt_block_raw(1, 2, 3, 4)*.

ElGamal: Key generation was traced with p = 23, g = 5, x = 6, giving y = 5^6 mod 23 = 8. Encryption of m = 10 with ephemeral key k = 3 produced ciphertext (c_1 = 10, c_2 = 14). The digital signature on m = 10 was computed step by step: k^{-1} mod 22 = 15, s = 15 × (10 − 6 × 10) mod 22 = 20, yielding signature (r = 10, s = 20). The verification equation was confirmed by hand.

Ring-LWE: Using n = 4 and q = 17, polynomial multiplication of a(x) = [3,5,2,7] and s(x) = [1,0,−1,1] was carried out coefficient by coefficient under the reduction $x^4 \equiv -1$, producing a·s = [6,0,6,5] mod 17. The public key b = a·s + e was assembled, and a single-bit encryption (m = 1) was traced to ciphertext u = [5,13,5,12], v = 3.

Performance Evaluation

The benchmarks/performance.py module measures computation and communication costs. The measurements include: encryption/decryption wall clock time, in MB/s for XTEA and IDEA at 1 KB, 10 KB, 100 KB payloads, ElGamal key-generation latency (256-bit prime), and peak memory via psutil. Communication cost is modelled as ciphertext to plaintext ratio, capturing overhead from the 8-byte CBC IV and PKCS#7 padding. The results appear in tables highlighting symmetric speed vs asymmetric security.

Testing

Testing used pytest at unit and integration levels. Each cipher was checked for exact encrypt–decrypt recovery, edge cases (empty, single-byte, block-size multiples), and raw block functions against handwritten results. The CA module was tested for issuance, verification, and rejection of revoked serials. ElGamal tests covered the full cycle: key generation, encrypting and decrypting integers, session-key roundtrips and signature verification, including making sure tampered data gets rejected. On top of that, the integration tests walked through the entire portal flow: registering a client, generating PQC tokens, doing the TLS-style handshake, uploading and downloading records, rotating session keys, and revoking certificates, with integrity checks at every step.

Post Quantum Cryptography

The Ring-LWE module was added as an optional extension, using polynomial arithmetic in $R_q = Z_q[x]/(x^n + 1)$ with n = 16, q = 257. HALF_Q (128) exceeds cumulative error (~33), ensuring reliable decryption. The module provides byte-level encryption and integrates as a quantum-resistant authentication layer: each client encrypts its ID under its Ring-LWE public key before handshake. Verification at n = 4, q = 17 validated multiplication and rounding before scaling to n = 16.

5. Use Case and Scenario Design

The project simulates a secure healthcare environment involving 3 parties: a client (healthcare employee), a server, and a Certificate authority.

The client recquires encrypted access to sensitive patient records using the healthcare portal.

The process is so that when a connection is initiated, the server provides a digital certificate, the client then verifies it through the Certificate Authority to confirm its legitimacy before any sensitive communication begins. A secure session is then established using ElGamal key exchange to distribute session keys between parties.

Once the session is active, IDEA or XTEA symmetric encryption protects transmitted healthcare data, chosen for their efficiency with large messages. Hashing and digital signatures additionally ensure message integrity and authentication.

The system follows a multi-layer security workflow:

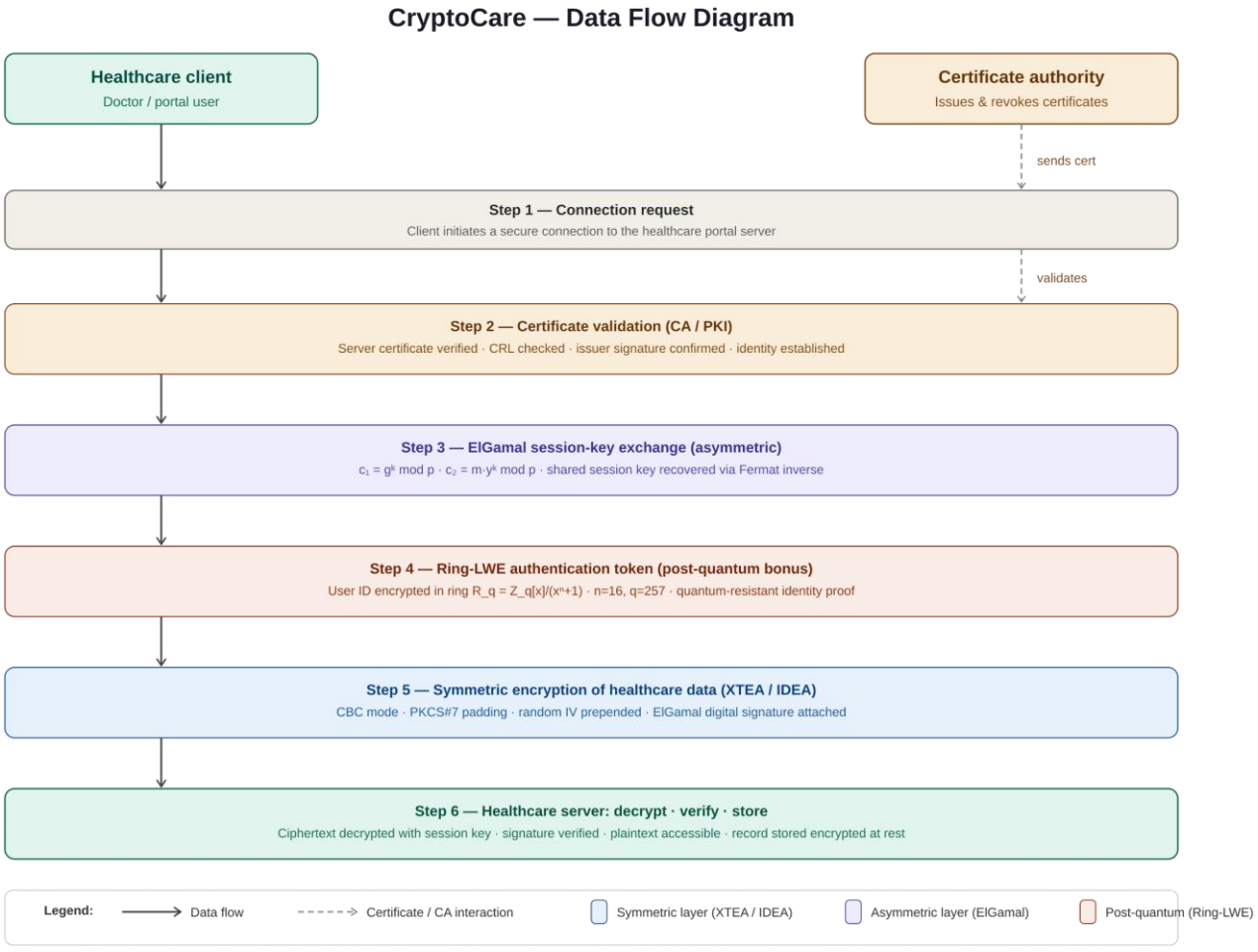
1. Client requests secure connection.
2. Server sends digital certificate.
3. Certificate Authority validates certificate authenticity.
4. ElGamal securely exchanges session keys.
5. IDEA/XTEA encrypt healthcare data.
6. Encrypted messages are transmitted securely.
7. Server decrypts and verifies received data.

The project also includes an optional post-quantum Ring-LWE module demonstrating how future-resistant methods may replace classical public-key systems in secure healthcare infrastructures.

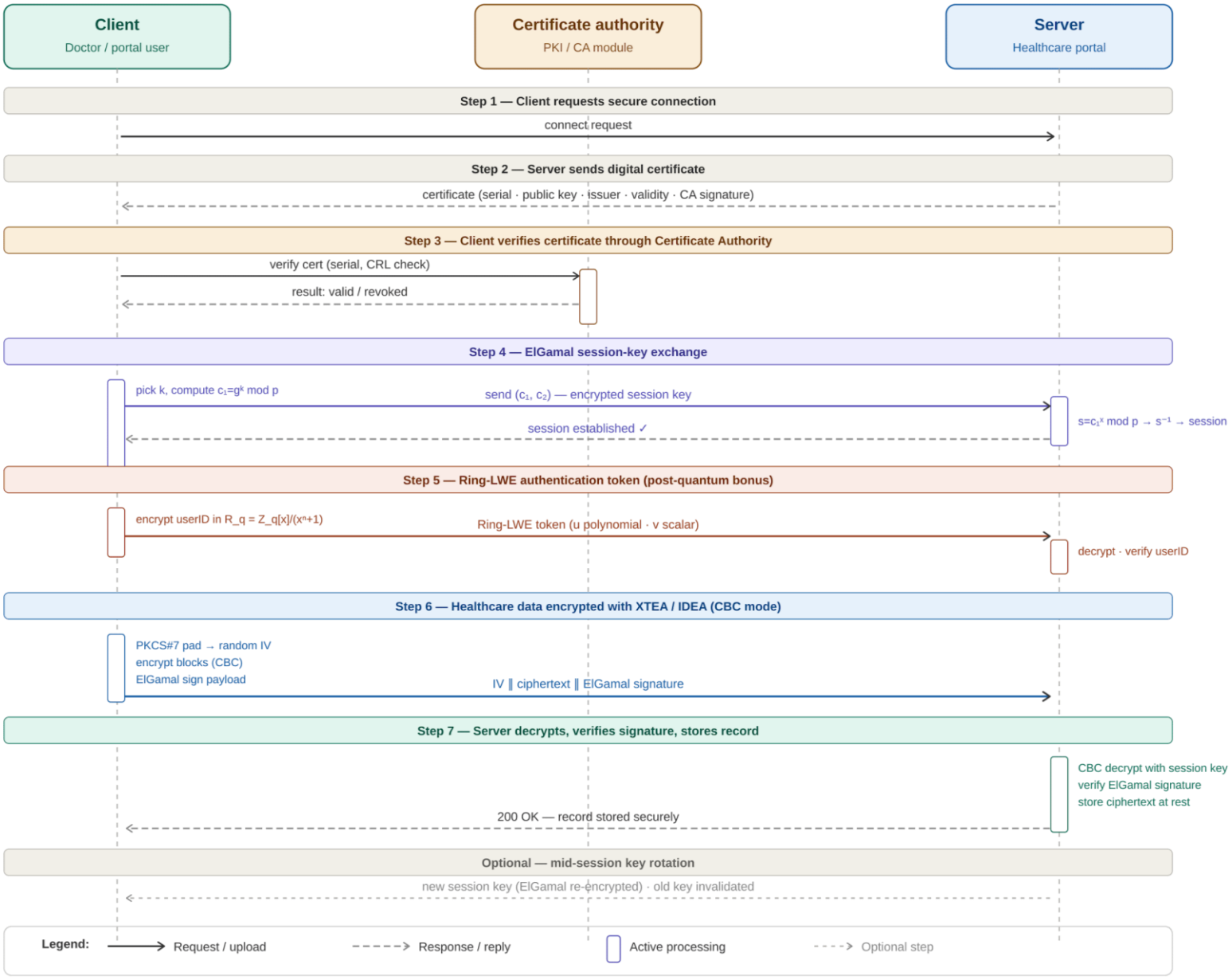
Key Generation and Distribution

The Symmetric keys for IDEA and XTEA are generated during session establishment and exchanged securely using ElGamal asymmetric encryption relying mainly on modular exponentiation and large prime numbers to produce public-private key pairs.

A Certificate Authority issues and validates the digital certificates, allowing clients to verify server identity before trusting the channel, preventing impersonation and man-in-the-middle attacks.



CryptoCare — Sequence Diagram



5. Conclusion

Building CryptoCare from scratch was more rewarding than expected. Implementing XTEA, IDEA, ElGamal, a certificate authority, and Ring-LWE without external libraries forced us to understand every mathematical step rather than relying on prebuilt functions, a depth of understanding that couldn't be gained otherwise.

The hybrid design makes the system practical and efficient as XTEA and IDEA efficiently encrypt bulk patient records in CBC mode while ElGamal handles session-key exchange and physician signatures to avoid asymmetric overhead.

The CA layer ensures that identities are verified, session-key rotation limits exposure from compromise and the Ring-LWE module shows awareness of post-quantum directions despite using toy parameters.

CryptoCare is proof that a secure healthcare communication system can be built from first principles. Scaling to a production level however, would require larger ElGamal primes and replacing Ring-LWE with CRYSTALS-Kyber which the architecture already anticipates.